

CSED211: Data Lab1 Report

컴퓨터공학과 20240940 장해영

1. Overview

주어진 연산자만을 이용하여 5가지의 연산을 수행하는 과제이다. if문과 같은 조건문없이 허용된 연산자만 이용할 수 있다.

2. Question #1. bitNor

A. Explanation

NOR연산은 OR연산($|$)과 NOT연산($\sim$)을 합친것이다. NOR연산을 적용할 때 먼저 OR연산을 적용한 후 NOT연산을 진행하므로 주어진 비트가 x, y 라 할 때 $\sim(x | y)$ 를 적용하면 NOR연산이 된다. 0과 1에 대해 가능한 NOR연산을 표로 정리하면 다음과 같다.

NOR	0	1
0	1	0
1	0	0

B. Solution

과제의 조건에 주어진 $\sim$ 와 $\&$ 만을 이용하여 NOR연산을 수행하기 위해 드모르간 법칙(De Morgan's Laws)을 이용하였다. 드모르간 법칙은 논리연산의 관계를 바꿀 수 있는 규칙이다. $\sim(x | y) \equiv (\sim x) \& (\sim y)$, $\sim(x \& y) \equiv (\sim x) | (\sim y)$ 총 두 가지로 표현할 수 있다. 두 가지 식 중 Nor 연산과 일치하는 첫번째 식을 이용하면 주어진 조건내에서 NOR연산을 표현할 수 있다.

C. Implementation

```
int bitNor(int x, int y) {  
    //to be implemented  
    return (~x) & (~y);  
}
```

새로운 변수를 만들 필요 없이, 바로 $(\sim x) \& (\sim y)$ 를 return하여 주어진 정수 2개에 대한 NOR연산을 수행하도록 하였다.

3. Question #2. isZero

A. Explanation

주어진 정수가 0이라면 1을, 0이 아니라면 0을 반환하는 과제이다. 복잡한 연산없이 주어진 정수만 0과 비교하면 된다.

B. Solution

C언어에서 NOT연산자(!)는 0이 아닌 값에는 0을, 0인 값에는 1을 반환하는 특성이 있다. 이러한 NOT연산자의 특징을 이용하면 문제를 쉽게 해결할 수 있다.

C. Implementation

```
int isZero(int x) {  
    //to be implemented  
    return !x;  
}
```

주어진 정수를 0과 비교하는 연산없이도 NOT연산자만을 적용하여 그 값을 return하면 문제에서 의도한 조건대로 isZero를 해결할 수 있다.

4. Question #3. addOK

A. Explanation

주어진 두 개의 정수를 더했을 때 overflow가 발생하면 0을, overflow가 발생하지 않으면 1을 반환하는 문제이다.

주어진 두 정수가 모두 양수이거나, 모두 음수인 경우 overflow가 일어날 가능성이 있다. 두 정수 중 하나는 음수, 하나는 양수인 경우, 두 수를 더하게 되면 주어진 양수보다 더 큰 양수가 된다거나, 주어진 음수보다 더 작은 음수가 되는 경우가 없기 때문에 overflow가 일어나지 않는다.

그렇다면, overflow가 발생할 가능성이 있는 경우에 대해 생각해보자. 주어진 두 정수가 모두 양수라 가정하자. 이 때 이 두 정수를 더한 값의 부호가 음수가 되면 overflow가 발생한 것이다. 반대로, 주어진 두 정수가 모두 음수인 경우, 이 두 정수를 더한 값의 부호가 양수이면 overflow가 발생한다.

위의 내용을 정리해보면, overflow가 발생하는 조건은 다음과 같다

1. 주어진 두 정수의 부호가 같을 것.
2. 주어진 두 정수를 더한 값과 주어진 정수의 부호가 다를 것.

위의 조건을 연산자로 표현하면 문제를 해결할 수 있다.

B. Solution

우리가 연산자로 나타내야하는 조건은 다음 두 가지 조건이다.

1. 주어진 두 정수의 부호가 같을 것.
2. 주어진 두 정수를 더한 값과 주어진 정수의 부호가 다를 것.

이 조건들을 표현하려면 우선 주어진 정수 x , y 의 부호를 판단해야 한다. 최상위 비트가 부호를 나타내는 비트이기 때문에 이 비트를 이용하면 부호를 알 수 있다. >> 연산자를 이용하면 C언어에서 arithmetic shift로 right shift되므로 모든 비트가 부호 비트로 채워져 부호를 판단할 수 있게 된다.

위와 같은 방법으로 부호정보를 담은 변수를 만들어 overflow의 조건에 맞는 지 판단하면 된다. 두 정수의 부호의 일치여부를 판단하기 위해서는 XOR연산자(^)를 사용하면 된다. XOR연산을 수행하면 비트가 일치하는 경우 0을, 일치하지 않는 경우 1을 반환한다. 0과 1이 주어졌을 때 XOR연산을 수행한 모든 경우는 다음 표와 같이 나타낼 수 있다.

XOR(^)	0	1
0	0	1
1	1	0

C. Implementation

```
int addOK(int x, int y) {
    //to be implemented
    int sum = x + y;

    int sign_x = x >> 31; // sign bit for x
    int sign_y = y >> 31; // sign bit for y

    int sign_sum = sum >> 31; // sign bit for sum
    return !((~(sign_x ^ sign_y)) & (sign_x ^ sign_sum));
}
```

주어진 정수인 x , y 와 이 두 정수를 더한 값인 sum 총 3개의 정수의 부호를 이용하여 overflow의 여부를 판단하였다. Overflow가 발생하는 조건인 두 정수의 부호가 같을 것, 주어진 정수와 더한 값의 부호가 다를 것 총 두 가지의 조건을 모두 만족하는 경우만 0을 return하고, 그렇지 않은 경우는 1을 return하도록 두 조건을 연결할 때 AND연산자(&)를 사용한 후 전체에 대해 NOT연산자(!)를 적용하였다.

5. Question #4. absVal

A. Explanation

주어진 정수 x 의 절댓값을 반환하는 문제이다. 주어진 정수 x 가 양수인 경우 그대로 출력하면 되지만, 음수인 경우 부호를 반전시켜 양수로 만든 후 출력해야한다.

B. Solution

주어진 정수를 절댓값으로 나타내기 위해 question #3과 같이 부호를 판단하는 작업이 필요하다. 위와 같이 부호비트만으로 이루어진 변수를 만들어 이를 연산에 사용한다. 아래에서 이 변수를 부호변수라 칭하였다.

다음으로 양수인 경우 값을 그대로 도출하면서, 음수인 경우 부호만 반전된 수를 반환하는 식이 필요하다. 이는 비트연산에서 음수를 표현하는 방법인 2의 보수를 이용하면 해결할 수 있다. 주어진 정수 x 의 부호를 반전시키는 방법은 우선 NOT연산($\sim$)을 통해 모든 비트를 뒤집은 다음, 1을 더해주는 것이다. 양수와 음수인 경우를 모두 만족시키기 위해 "① 모든 비트를 뒤집기 ② 1을 더해주기" 두 단계로 나누어 연산을 수행하면 된다.

① 모든 비트 뒤집기

음수인 경우에만 비트를 뒤집기 위해 XOR연산을 사용하면 된다. 주어진 정수 x 가 양수인 경우 부호변수의 비트가 모두 0, 음수인 경우는 모두 1인 점을 이용하여 x 와 부호변수에 XOR연산을 적용하면, 양수인 경우는 그대로 x 가, 음수인 경우는 모든 비트가 반전된 수가 도출된다.

	부호변수	X	부호변수 ^ X
양수	0x00000000	X	X
음수	0xFFFFFFFF	X	$\sim X$

② 1을 더해주기

이 연산도 마찬가지로 음수인 경우만 1을 더해주기 위해 부호변수를 이용한다. NOT연산자($\sim$)를 이용하여 부호변수의 비트를 모두 반전하면, 양수인 경우 모든 비트가 1, 음수인 경우 모든 비트가 0이 된다. 이 상황에서 1을 더해주면 양수의 경우 overflow가 발생해 0이, 음수인 경우 1이 더해지는 결과를 끌어낼 수 있다.

	부호변수	$\sim$ 부호변수	$\sim$ 부호변수+1
양수	0x00000000	0xFFFFFFFF	0x00000000
음수	0xFFFFFFFF	0x00000000	0x00000001

이 모든 결과를 종합하여 ①, ②를 더해주면 주어진 정수의 절댓값을 반환할 수 있다.

C. Implementation

```
int absVal(int x) {  
    //to be implemented  
    int sign_x = x >> 31;  
    return (x ^ sign_x) + (~sign_x + 1);  
}
```

Sign_x를 부호변수로 이용하여 위 solution의 과정을 수행하였다.

6. Question #5. logicalShift

A. Explanation

C언어에서 right shift를 수행하면 기본적으로 arithmeticShift가 수행된다. arithmeticShift란, right shift를 할 때 최상위 비트를 그대로 복사하여 채우는 방식을 말한다. 반면, logicalShift는 최상위 비트의 종류와 상관없이 무조건 0을 채우는 방식이다.

예를 들어, 양수인 5를 의미하는 00000101를 오른쪽으로 두 칸 shift하면 arithmeticShift와 logicalShift에 상관없이 00000001이 된다. 반면, 음수인 -8을 의미하는 11111000을 오른쪽으로 두 칸 arithmeticShift하면 11111110이지만, logicalShift하면 00111110이다.

B. Solution

C언어에서 right shift를 하면 기본적으로 arithmeticShift가 수행되므로 최상위비트를 따라 채워진 비트를 모두 0으로 변경하는 작업이 필요하다. 이를 위해 mask변수를 도입하여 AND연산(&)을 진행하면 된다. Shift를 n칸 진행한다고 했을 때 mask는 상위 n개의 비트가 0, 나머지는 모두 1인 변수로 설정하면 된다. 반대로 생각하면 하위 (32-n)개의 비트가 1이면 되기 때문에 mask변수를 $\sim(\sim 0 \ll (32 - n))$ 와 같이 설정하면 된다.

0	~ 0	$\sim 0 \ll (32-n)$	$\sim(\sim 0 \ll (32-n))$
0000...0000	1111...1111	1111...0000 (1 -> n개)	0000...1111 (0 -> n개)

이렇게 만들어진 mask변수와 arithmeticShift를 수행한 주어진 정수 x에 대해 AND연산(&)을 진행하면 최종적으로 logicalShift를 수행한 값을 반환하게 된다.

C. Implementation

```
int logicalShift(int x, int n) {  
    //to be implemented  
    int mask =  $\sim(\sim 0 \ll (31 + (\sim n + 1)) \ll 1)$ ;  
    int arithShift_x = x >> n;  
    return arithShift_x & mask;  
}
```

위의 solution에서는 $\sim(\sim 0 \ll (32 - n))$ 식을 이용한다고 하였지만, 실제코드에서는 -를 이용할 수 없고, n이 0인경우에 shift를 제대로 수행할 수 없기에 -n을 $(\sim n + 1)$ 로 바꾸고, 하나의 비트를 나중에 연산하는 것으로 문제를 해결하였다.